Summary of "An Extreme Learning Machine-Based Method for Computational PDEs in Higher Dimensions"

Prepared by ChatGPT

May 1, 2025

1 Overview

The paper by Y. Wang and S. Dong (Computer Methods in Applied Mechanics and Engineering, 418:116578, 2024) proposes two *randomised neural network* solvers for high–dimensional partial differential equations (PDEs):

1. **Extreme Learning Machine (ELM)** with fixed hidden–layer parameters;
2. **ELM/A–TFC**, which embeds an "Approximate" Theory of Functional Connections (A–TFC) constraint to enforce boundary/initial conditions exactly while leaving a free field represented by an ELM network.

Both techniques transform the PDE residual at randomly sampled collocation points into a (non-)linear least–squares system for the *output* weights, avoiding gradient-based training on deep architectures.

2 Algorithmic Details

2.1 Notation

Let $\Omega \subset \mathbb{R}^d$ be a rectangular domain and consider the generic boundary-value (or initial–boundary-value) problem

$$\mathcal{L}u(x) + \mu\mathcal{N}(u(x)) = Q(x), \qquad x \in \Omega, \tag{1}$$

$$\mathcal{B}u(x) = H(x), \qquad x \in \partial\Omega, \tag{2}$$

with possibly time-dependent operators. Here $\mathcal{L}$ is linear, $\mathcal{N}$ may be nonlinear, and $\mathcal{B}$ represents boundary/initial conditions.

2.2 ELM solver

1. **Random projection.** Choose the activation $g(\cdot)$ (e.g. tanh or sin). Draw weights $w_j \in [-R_m, R_m]^d$ and biases $b_j \in [-R_m, R_m]$ independently and fix them forever.
2. **Ansatz.** Represent the solution by a single-hidden-layer network $u_M(x) = \sum_{j=1}^{M} a_j\, g(w_j{\cdot}x + b_j)$, where only the output weights a_j are trainable.
3. **Collocation sampling.** Generate
   - N_{in} interior points $\{x^{(k)}\}$,
   - N_{bc} boundary points $\{x_\partial^{(k)}\}$ (and N_{t_0} initial points for parabolic PDEs).

4. **Residual assembly.**

$$\begin{aligned}\text{Interior:} \quad & r^{(k)} = \mathcal{L}u_M(x^{(k)}) + \mu\mathcal{N}(u_M(x^{(k)})) - Q(x^{(k)}),\\ \text{Boundary:} \quad & b^{(k)} = \mathcal{B}u_M(x_\partial^{(k)}) - H(x_\partial^{(k)}).\end{aligned}$$

5. **Least-squares training.** Form the stacked residual vector $F(a) \in \mathbb{R}^N$ ($N = N_{\text{in}} + N_{\text{bc}}$).

   - *Linear PDE*: $F(a) = Aa - q$ is linear in a; solve $a^\star = \arg\min \|Aa - q\|_2$ via pseudo-inverse.
   - *Nonlinear PDE*: apply Levenberg–Marquardt to $\min_a \|F(a)\|_2^2$ (typically converges within $\mathcal{O}(10)$ iterations owing to small parameter count).

6. **Prediction.** The trained network $u_M(x; a^\star)$ approximates $u(x)$ everywhere.

2.3 ELM/A–TFC solver

1. Construct an *A–TFC constrained expression* $u(x) = G(x) + \mathcal{C}[f](x)$, where G is a simple function satisfying the non-homogeneous boundary data and $\mathcal{C}[\,\cdot\,]$ embeds homogeneous boundary conditions.

2. Approximate the free function $f(x)$ by an ELM network with coefficients α_j.

3. Follow steps 3–5 above to train α_j. By construction the final $u(x)$ exactly satisfies the boundary/initial conditions.

3 Theoretical Findings

Building on the work of Rahimi & Barron and, specifically, the result by Igelnik & Pao (1995), the authors recall that for any Lipschitz f on $[0,1]^d$ there exists a random-feature expansion with *expected* L^2 error $\mathbb{E}\,\|f - f_M\|_{L^2} \le C\,M^{-1/2}$, where C depends on the Lipschitz constant but *not* on the dimension d. This dimension-independent $M^{-1/2}$ rate motivates using large but *shallow* random networks for high-dimensional PDEs.

4 Numerical Results

The paper reports extensive experiments for dimensions $d = 3$–10 using networks of width $M = O(10^3)$. Below we list the headline results:

- **Poisson equation** on $[-1,1]^d$ ($d = 3,5,7,9$): ELM achieved L^∞ errors from 2.3×10^{-10} (3-D) to 4.0×10^{-5} (9-D) with training times 2.6–14.6 s—*two orders of magnitude faster and* 10^2–10^4 *times more accurate* than a comparable PINN (Table 16).

- **Non-linear Poisson**: similar trends; ELM errors $\le 1.6 \times 10^{-4}$ (9-D) vs. PINN $\approx 5 \times 10^{-2}$, with $\sim 10\times$ shorter training (Table 17).

- **Advection–diffusion ($d = 3,6,10$)**: exponential error decay w.r.t. hidden width until saturation; max error $< 10^{-7}$ for $d = 3$ (Fig. 11). Optimal sampling radius R_m decreases with d (Tables 5–6).

- **Heat equation ($d = 7$)** solved by *ELM/A–TFC*: point-wise errors $\mathcal{O}(10^{-4})$ and exponential convergence versus boundary points (Figs. 24–25).

Overall, the ELM family delivers PINN-level (or better) accuracy at drastically lower computational cost, and the dimensional curse is mitigated but not entirely removed.

5 Conclusion

Randomised, shallow ELM networks—trained by one-shot least-squares—constitute a simple yet powerful alternative to gradient-based deep PINNs for high-dimensional PDEs. The optional A–TFC embedding further improves boundary fidelity at modest extra cost.